

Account Maintenance

Emptying scratch files, refreshing a VOC, keeping local versions of records, configuration, the system date, and deleting an account.

This page continues Accounts and Security.

Emptying an account's scratch files: `clean.account`

```
clean.account
```

Empties three things in the account you are standing in, and takes no arguments — **there is no way to clean an account you are not in:**

```
:clean.account
Cleaned $COMO
Cleaned $hold
Cleaned $savedlists
```

<code>\$COMO</code>	captured session transcripts, including every phantom's
<code>\$hold</code>	reports sent to the hold file instead of a printer
<code>\$savedlists</code>	saved select lists

Nothing else is touched — no data file, no program, no dictionary. A como capture that is currently running is left alone and says so: *\$COMO not cleaned - COMO file active*.

It needs no elevation. It is the one verb here an administrator account can use from an ordinary session, which is right: it deletes only that account's own scratch.

Refreshing an account's VOC: `update.accounts`

```
update.accounts {all}
```

```
Copying records from NEWVOC to VOC...
```

Copies the shipped verb and keyword definitions into an account, adding what is missing and leaving that account's own VOC entries alone. It is what brings an existing account up to date after SD itself is upgraded, and **it respects the account's tier** — a standard account does not collect programmer verbs by being refreshed, and a suspended one keeps the tier it was suspended from.

With no keyword it updates the account you are standing in and then offers the rest, asking each time. `all` is the **unattended form**: it updates every registered account without asking.

You will not normally type `all` yourself. The installer runs it during an upgrade, so a release that adds a VOC record reaches every existing account without anybody visiting them. Before that existed, an upgrade replaced the shipped files and no account gained a new verb.

`all` is an explicit keyword rather than something inferred from how SD was started. The test for an internal session exists and would have worked, but it would have decided a rewrite of every account's VOC from a property nobody typing the verb can see.

A second word that is not `all` is refused by name rather than ignored:

```
:update.accounts everything
UPDATE.ACCOUNTS does not take everything
```

Quietly ignoring it would run the interactive form while the caller believed they had asked for the other one.

IT NEVER TAKES ANYTHING AWAY

`update.accounts` only ever adds. A record removed from an account stays removed. To keep an edit of your own, mark the record — see below.

Keeping your own version of a VOC record: `[locked]`

This has no equivalent in OpenQM or in SD on Linux. Nothing you know from another MultiValue system will tell you it exists.

A site that has customised one of SD's own VOC records marks it by putting `[locked]` in **field 1, after the type code**.

`update.accounts` then leaves that record alone.

```
V[locked]
CA
$MYVERSION
```

AFTER THE TYPE CODE, NOT AT THE FRONT

The first character of field 1 **is** the type, so `[locked]V` would be read as a record of type `[`. Two characters are the type for a `P` record, and those are the only two-character types SD uses: `PA` for a paragraph and `PH` for a phrase.

```
PA[locked]
```

Anywhere later in field 1 works — the test searches the field rather than matching a fixed position. Case does not matter: both sides are upper-cased before comparison, so `[LOCKED]` and `[Locked]` are the same marker. A lock that failed open because somebody typed it in capitals would be worse than no lock at all, because the record it was meant to protect would be replaced silently.

A VERB IS NOT PROTECTED BY IT

`[locked]` **is honoured on every kind of VOC record except a verb.** A verb marked `[locked]` is updated anyway, and the account is told which ones:

```
2 verb(s) marked [locked] were updated anyway: MYLIST MYREPORT
```

A verb is what SD runs. A locked one would go on naming the program, or the internal routine number, that this release replaced — and the internal case fails in the worst way available, because field 3 of an internal verb record is a *number*, so a reassignment sends the verb to a different function with no sign that anything is wrong.

The way to get a verb that behaves differently is to add your own, under a name SD does not ship, modelled on the system one. `update.accounts` walks the records SD ships, so a verb of your own is never visited and needs no marker.

YOU ARE TOLD WHAT WAS WITHHELD

```
3 VOC record(s) were left alone because they are marked [locked]: WINDOWS ...
```

The message names each record, and says plainly that any correction this release made to them has not been applied here.

That is the cost, and it belongs to the site rather than to SD. A locked record keeps the version the account already had, so a defect fixed in that record stays unfixed in this account. Remove the marker from field 1 and run

```
update.accounts
```

 again to take the new version.

Reading and setting configuration: `config`

```
config          report every setting
config lpnr     the same, to the default printer
config param value set one
config gpl      display the licence
config contrib  display the contributors
```

```
:config
Virtual Machine Version Number W1.0-0
APILOGIN 1
APIPORT 4243
CMDSTACK 99
DEADLOCK 0
DUMPDIR
ERRLOG 50 kb
...
NUMFILES 80
NUMLOCKS 100
NUMUSERS 20
...
SH C:/Windows/System32/WindowsPowerShell/v1.0/powershell.exe -NoProfile -NoLogo
SORTWORK /cygdrive/c/WINDOWS/TEMP
TMPDIR /cygdrive/c/WINDOWS/TEMP
YEARBASE 1930
```

(Forty-odd lines; cut here.) **Reading needs nothing** — any account with the verb can do it, and it is the quickest answer to *how many users, how many locks, how many open files is this machine set up for*.

New parameter value required

`config numlocks` with nothing after it. **The report form is `config` alone**; naming one parameter means *set it*

Not a recognised private configuration parameter name

the name is not one that can be set per-session

Invalid value for this parameter

it is, and the value is not

`config param value` **sets a private, session-local value and not the machine's**. The machine's settings live in SD's configuration file and are read when SD starts. This form overrides one for the session you are in, which is the right tool for trying a value before writing it down and the wrong one for changing an installation.

`config gpl` and `config contrib` **read a record inside SD** rather than running a pager over a file, so they work in any account and need no operating-system access.

Setting the machine's date: `set.date`

```
set.date date
```

Sets the **machine's** date, not a session preference — it changes the clock the whole installation reads. The argument goes through SD's `D` conversion, so anything `iconv(..., 'D')` accepts will do, and anything it does not is refused:

Date required

`set.date` with nothing after it

Invalid date format

the argument is not a date SD can read

There is no confirmation and no undo. It is described here from source rather than shown running, because demonstrating it would move the clock of whatever machine it ran on. **On Windows, changing the system date is itself a privileged operation**, so a session that has the verb may still be refused by the operating system underneath it.

Moving a live machine's date backwards is not a neutral act: file timestamps, licence expiry, scheduled tasks and anything that reasons about elapsed time all read it. Treat it as a maintenance operation on a quiet system.

Deleting an account: `delete.account`

```
delete.account account.name
```

Removes the account directory, its Windows group, its entry in the accounts register, and — for a user account SD itself created — the Windows account and its profile. **One confirmation covers all of it**, and the wording is decided before the question is asked, so it never offers to remove a Windows account it is not going to.

It will not delete a Windows account SD did not create. The account is left in place and it says so.

Three refusals come before the confirmation, so none of them can be reached by accident:

Cannot delete SDSYS account

`delete.account sdsys`

Cannot delete own account

the account you are standing in

Account not registered in ACCOUNTS file

the name is not one of SD's

(Those three wordings are the verb's own; they are not shown as a transcript here because reaching them takes an elevated session, and an unelevated one is refused by the privilege gate first — which is itself the fourth refusal, and the one most people meet.)

The confirmation is unconditional and no keyword suppresses it. There is no `no.query` on this verb. **Never send `delete.account` down a pipe** — the prompt will eat the commands that follow it as its answers, and the session will then wait for ever.

Who has these verbs

All of them are administrator verbs. A standard or programmer account has none of these names at all.

needs elevation as well

`create.account` `modify.account` `modify.password` (for another account)
`delete.account` `grant` `revoke` `list.grants`

the verb is enough

`clean.account` `update.accounts` `config`

`list.grants` **needs elevation even though it only reads.** It answers *who may enter this account*, which is worth knowing before you have it, and the gate is at the top of the program the three grant verbs share.

See also

Sessions and Locks · Operating System Access.

In the user documentation, which does not repeat any of this: *SD TCL - The Command Processor* for how a verb is dispatched and what a VOC record holds, and *SD Basic - System and Environment* for what a program can read about its own session. Those pages are in a different set and are deliberately not linked from here — see the note at the top.